

KLA FLEETPACK · NEXUS THEME V2

FleetPack FAM

Fleet Availability Module — Complete Technical Documentation

Folder Structure · Architecture · Base Module · Storybook · Shared Module · Core · Features

Angular 20 · Signals · Tailwind CSS 4 · Chart.js 4 · Storybook 10 · Mock REST API

Generated documentation — consolidates README, component reference and architecture guide.

26 August 2026

Table of Contents

1. Overview & Quick Start

2. Full Folder Structure

3. End-to-End Data Flow

4. The Base Module — Reusable Component Library (base/)

4.1 Components

4.2 Storybook — the interactive component catalog

4.3 Table Model (models/table.model.ts)

4.4 Column filters & Manage Columns

4.5 Expandable nested child tables

4.6 Template directives

4.7 Date pickers

4.8 Used Throughout the App

4.9 Full component reference

5. The Shared Module — App-Level Composition Layer (shared/)

6. The Core Module — Data & Infrastructure Layer (core/)

7. Layout — The App Shell (layout/)

8. Features & Routing (features/)

9. Reference — Domain Models

10. Adding a New Module — Checklist

1. Overview & Quick Start

Angular implementation of the FAM (Fleet Availability Module) screens from the Nexus Theme v2 design (KLA FleetPack). Standalone components, signal-based state management, Chart.js visualizations, custom heatmap/Gantt renderers, and a fully mocked REST API — no backend required.

```
npm install
npm start          # ng serve --open → http://localhost:4200
```

Requires Node 18.19+, 20.11+, or 22.0+ (Angular 20). Stack: Angular 20 + @angular/cdk · Tailwind CSS 4 (CSS-first @theme tokens) · Chart.js 4 + ng2-charts 6 · RxJS 7 · signals for all state (no NgRx/Akita) · Storybook 10 (@storybook/angular, addon-a11y, addon-docs) for isolated component review.

Screens & Routes

Route	Screen
/fleet-availability/up-time/analysis	Fleet Up+Time Analysis — Uptime/Downtime Trend toggle on the rolling line chart (1W/13W + period average), collapsible uptime breakdown table (rolling, tool-wise, grouped by SW version) with chip-style Tool-wise/Grouped toggles + CSV export, Top-10 unavailable tools stacked bar, downtime category details with colored progress bars.
/fleet-availability/up-time/availability	Fleet Up+Time Availability — single divided KPI strip (13W/4W rolling, current week, MTBr, total downtime in red), Uptime/Downtime Trend line chart, Top-10 unavailable stacked bar with a Non-Scheduled toggle, downtime categories (colored label + underline bar), a collapsible Tool-Level Analysis Filter bar (tool picker + live state-count pills).
.../availability/heatmap	Tab 1 — 24-Hour State Heatmap (custom CSS-grid, 14 days × hour blocks; state totals live in the page-level Tool-Level Analysis Filter bar).
.../availability/gantt	Tab 2 — Activity Gantt (Sys E10 / Tool E10 rows per day, segment labels, colored availability badges, "Day Shift" as a legend swatch, pagination in the footer).
.../availability/events	Tab 3 — Event Details (grouped-by-day table with sortable timestamps, progress-bar duration, outlined state badges, delete/edit row actions, paging, Add Event, CSV).
/alarm-explorer	Alarm Explorer home — Model/Fleet/Category/Duration filter bar, stacked alarm-volume chart by category (Trend/Pareto tabs), fleet breakdown drill-down list.
/alarm-explorer/fleet/:fleetId	Fleet detail — Alarm ID/Tools/Category/SW Version filter bar, 5-metric KPI strip, tool-level stacked distribution (Trend/Pareto tabs, click a bar to drill down), top alarms with category-colored bars, tool summary table.
/alarm-explorer/fleet/:fleetId/tool/:toolId	Tool alarm summary — Alarm ID/Category/Severity/Recipe filter bar + searchable alarm table; selecting a row opens the Alarm Info inspector in a slide-over <base-drawer>.
.../tool/:toolId/alarm/:alarmId	Alarm events — Date Range/Recipe/Show/Duration filter bar, 4-metric KPI strip, chronological event log with a recipe tab filter, outlined resolution badges, pagination, CSV.
/fleet-configuration, /fleet-productivity, /tqual, /my-reports, /innovation-lab, /engineering-utilities	Placeholder modules wired into nav & routing.

Layering rule

features/ depends on shared/ and core/; shared/ depends on base/ and core/; base/ depends on nothing app-specific (no chart.js, no models, no services) — it is a self-contained, exportable design-system library, cataloged twice over: the in-app /dev/base playground and the standalone Storybook build.

2. Full Folder Structure

```
fleetpack-base-module/
├─ angular.json           Angular CLI workspace config
├─ package.json           deps: @angular/*, chart.js, ng2-charts, rxjs, tailwindcss, storybook
├─ tsconfig.json / tsconfig.app.json
├─ postcss.config.js / .postcssrc.json  Tailwind 4 via PostCSS
├─ .storybook/           Storybook config: main.ts, preview.ts, tsconfig.*.json
├─ README.md             top-level quick start + architecture summary
├─ docs/
│   └─ FleetPack-FAM-Documentation.pdf  this document
│   └─ Base-Module-Component-Guide.docx  full prop/event reference for every base-* component
├─ src/
│   └─ main.ts            bootstrapApplication(AppComponent, appConfig)
│   └─ index.html
│   └─ styles.css         Tailwind 4 @theme tokens + component classes (panel, btn-*, table-th, chip-toggle, ...)
│   └─ stories/           Storybook stories – one file per component/widget, mirrors src/app/base +
├─ shared/dynamic
│   └─ foundations/       Color, Typography, Spacing, Surfaces/Radius/Elevation, Grid, Iconography,
│   │                     Density & Themes, Token Architecture, Introduction.mdx
│   └─ base/*.stories.ts  one story file per base-* component (71 files)
│   └─ widgets/*.stories.ts  chart/kpi-grid/ranked-list/table/dynamic-page widget stories
│   └─ layout/sidebar.stories.ts
│   └─ shared/{Deprecated.mdx, state-legend.stories.ts}
├─ app/
│   └─ app.component.ts   root shell – just <router-outlet>
│   └─ app.config.ts      providers: router, http (+interceptors), charts, widget bootstrap
│   └─ app.routes.ts      route tree (see §8)
│   └─ base/              REUSABLE COMPONENT LIBRARY (design-system layer)
│       └─ index.ts        barrel export – public API of the module
│       └─ README.md       module overview + quick-start snippets
│       └─ models/table.model.ts  BaseColumnDef (30+ fields), 18 cell kinds, 17-type row-action
├─ registry, filter/page/sort/scroll/edit events
│   └─ directives/cell-template.directive.ts  baseCell, baseChildCell, baseHeaderCell,
baseActionTemplate, baseChildFooter
│   └─ utils/table-cell.utils.ts             cell formatting + summary-aggregate helpers shared by base-table
+ base-table-cell
│   └─ services/base-theme.service.ts        BaseThemeService – Light/Dark/High-contrast theme switching
│   └─ services/base-density.service.ts      BaseDensityService – app-wide default table density
│   └─ components/
│       └─ table/                          table-family components
│           └─ base-table.component.ts      <base-table> – the core data grid
│           └─ base-table-cell.component.ts <base-table-cell> – per-kind cell renderer
│           └─ base-table-views.component.ts <base-table-views> – saved-views tab rail
│           └─ base-paginator.component.ts  <base-paginator>, <base-search-input>
│           └─ base-manage-columns.component.ts <base-manage-columns> – show/hide + drag reorder panel
│           └─ base-column-filters.components.ts <base-checkbox-filter>, <base-calendar-filter>, <base-
range-filter>
│   └─ base-ui.components.ts                badge, tag, chip, avatar (+group), trend, KPI card, stat
bar,
│   └─ base-form.components.ts              button, split button, segmented control, button group,
│                                           text input, textarea, select, checkbox (+group), radio
│                                           group, toggle, selection cards, numeric stepper, OTP
│                                           input, color picker, time picker
│   └─ base-advanced-form.components.ts     combobox, multi-select chips, file upload, slider, range
slider
│   └─ base-datepicker.component.ts         <base-datepicker> – optional time-of-day (showTime) mode
│   └─ base-date-range-picker.component.ts  <base-date-range-picker>
│   └─ base-nav.components.ts               breadcrumbs, tabs, stepper, dropdown menu, context menu
│   └─ base-notification.components.ts      notifications panel, global search
│   └─ base-overlay.components.ts           modal, popover, hover card, [baseTooltip], [baseTeleport],
│                                           alert, banner, progress bar, skeleton, error page, toast
│   └─ base-drawer.component.ts             <base-drawer> – slide-over side panel
│   └─ base-charts.components.ts            trend/bar/scatter charts, histogram, chart frame
```

- └─ base-timeline.components.ts state heatmap, Gantt timeline, machine-state color scale
- └─ shared/ APP-LEVEL COMPOSITION LAYER (built ON TOP of base/)
 - └─ components/ui.components.ts fam-kpi / fam-loading / fam-trend (deprecated) + fam-state-legend
 - └─ dynamic/ the dynamic/config-driven widget system
 - └─ utils/ csv.util.ts, category-colors.util.ts
- └─ core/ DATA / INFRASTRUCTURE LAYER
- └─ layout/ App shell (topbar hosts the density/theme switchers)
- └─ features/ Screens = header + WidgetConfig[]

3. End-to-End Data Flow

```
Component (features/**)
  | reads signals from, calls actions on
  ▼
Store (core/state/*.store.ts – createQuery/createPagination/createListFilter)
  | calls
  ▼
ApiService (core/services/api.service.ts – typed HttpClient wrapper)
  | HTTP GET /api/**
  ▼
authInterceptor → mockApiInterceptor (core/mock-api/mock-api.interceptor.ts)
  | matches path → generator, adds latency
  ▼
mock-data.ts (build...()) – deterministic seeded fake data
```

A component never talks to HttpClient or mock data directly — only to a store. That indirection is what makes the mock-to-real-backend swap a one-file change (remove mockApiInterceptor from app.config.ts). For rendering, a component composes WidgetConfig[] from store data and hands it to <fam-dynamic-page>, which dispatches each widget to a renderer built from base/ primitives.

Layer	File(s)	Responsibility
Models	core/models/models.ts	Typed response/row shapes — the contract between mock and real API
Mock data	core/mock-api/mock-data.ts	build...() generators producing deterministic fake payloads
Mock routing	core/mock-api/mock-api.interceptor.ts	Matches /api/** paths to a generator, adds artificial latency
Service	core/services/api.service.ts	One typed method per endpoint, returns Observable<T>
Store	core/state/*.store.ts	Signal-based state: loading/error/data, caching, pagination
Component	features/**	Template + computed() widget configs, no data-fetching logic

4. The Base Module — Reusable Component Library (base/)

Self-contained, dependency-free (no chart.js required) set of standalone Angular components. Everything uses signal inputs (props) and typed outputs (event listeners), OnPush change detection, and the new control-flow syntax.

Live demo route: /dev/base. Every component is also cataloged one at a time in Storybook (npm run storybook) — see §4.2 — and listed live at /dev/components.

Everything is re-exported from one barrel, src/app/base/index.ts — always import via import { ... } from '.././base', never reach into a component file directly.

4.1 Components

Grouped the same way as the Component Catalog (/dev/components) and the Storybook sidebar. The table-family components live under components/table/; everything else lives flat in components/.

Selector	File	Purpose
Actions		
<base-button>		Primary command surface — primary/secondary/tertiary/ghost/destructive variants, loading + disabled states.
<base-split-button>		A default action plus a chevron menu of closely related variants.
<base-segmented-control>		A closed set of 2–4 mutually exclusive view options that changes what's shown without navigating.
<base-button-group>		2–4 related actions that share one bordered silhouette but fire independently.
Forms		
<base-text-input>		Text/number/password input: label, hint, error, warning, success, prefix/suffix, clearable, loading, read-only.
<base-textarea>		Multiline input with character counter.
<base-select>		Custom dropdown with optional search; [showChevron]="false" renders it as a plain input-styled box.
<base-checkbox>		Checkbox with indeterminate support, optional description text.
<base-checkbox-group>		Multi-select where every option stays visible at once — no dropdown, no chips.
<base-radio-group>		Radio button group, horizontal or vertical.
<base-toggle>		On/off switch, applies immediately.
<base-combobox>		Type-ahead field where the typed value is real, not just a filter; supports grouped options.
<base-multi-select-chips>		Multi-select field rendering selected values as removable inline chips.
<base-selection-cards>		One of N, presented as cards (label + description + icon) instead of a radio-group's dot+label row.
<base-file-upload>		Drag-and-drop upload zone with a per-file progress row; field-level error/warning tint matches base-text-input.

Selector	File	Purpose
<base-slider>		Bounded numeric range slider with a visible current-value bubble.
<base-range-slider>		Dual-handle numeric range slider — a From/To band instead of a single value.
<base-numeric-stepper>		Bounded integer entry via decrement/value/increment.
<base-otp-input>		Fixed-length numeric code, one digit per box; auto-advances on entry.
<base-color-picker>		Restricted to the design system's own token palette — a swatch picker, not free-form hex entry.
<base-time-picker>		Dropdown list of preset time slots — pick one, don't type one.
<base-datepicker>		Popup calendar: min/max, disabled-date rule, clearable, optional showTime HH:MM boxes.
<base-date-range-picker>		Dropdown: quick-range sidebar (All / Last 1-7-30 Days / Custom range) + dual month calendars with per-side HH:MM time boxes, Cancel/Apply.
Data display		
<base-badge>		States a fixed-vocabulary fact; never interactive.
<base-tag>		Static, non-removable classifier.
<base-chip>		Removable, user-applied filter/choice chip.
<base-avatar>		Person identity chip — initials on a deterministic tint, sm/md/lg/xl.
<base-avatar-group>		Overlapping avatar stack with a "+N" overflow badge.
<base-trend>		▲/▼ percentage pill; color flips via badWhenUp; zero renders "No change".
<base-kpi-card>		Label/value/unit/trend/sub KPI tile, optionally clickable/selected, with a partial-failure error+retry state.
<base-stat-bar>		Borderless horizontal row of metrics for a page header.
<base-sparkline>		Inline SVG mini chart (also the table's sparkline cell kind).
<base-list-item>		Full-row click target with hairline divider.
<base-accordion>		Collapsible content section with a chevron toggle.
<base-card>		Generic content card shell — icon/title header, projected body, footer row.
<base-divider>		Plain or labeled horizontal rule.
<base-loading>		Indeterminate spinner or dot variant, with a compact mode.
<base-skeleton>		Loading placeholder shaped like its content.
<base-empty-state>		no-results/no-access/no-data/out-of-range/not-configured/custom placeholder with an optional CTA.
<base-error-page>		Full-panel 404/403/500/offline state with a copy-to-clipboard trace id.
Navigation		
<base-breadcrumbs>		Navigation trail (routerLink or click events).
<base-tabs>		Headless tab bar — underline, pills, or vertical.
<base-stepper>		Linear progress stepper (horizontal or vertical), status derived from [(activeId)]'s position in [steps].

Selector	File	Purpose
<base-dropdown-menu>		Actions menu with icons, dividers, danger items, keyboard shortcuts.
<base-context-menu>		Right-click/overflow-triggered menu; opened imperatively via openAt(x, y).
<base-notifications-panel>		Header-anchored notifications overlay with unread state.
<base-global-search>		Header-anchored command-style global search.
Overlay & feedback		
<base-modal>		Content-projected dialog; sm/md/lg/xl/full sizes, icon header, destructive mode.
<base-drawer>		Content-projected slide-over panel: left/right/bottom side, configurable width, backdrop dim, footer slot.
<base-popover>		Anchored panel for interactive content, with focus-trapping.
<base-hover-card>		Hover-triggered rich preview of an entity, reachable from a dense table without leaving it.
<base-alert>		Persistent, section-scoped info/success/warning/error/neutral banner.
<base-banner>		Page-wide condition banner — use base-alert instead for one region.
<base-progress-bar>		Determinate progress, or [indeterminate]="true" for an unknown-duration operation.
<base-toast-host>		Renders the transient, auto-dismissing toast stack from BaseToastService.
[baseTooltip]		Hover/focus tooltip directive for any element; [tooltipTitle] switches to a richer title+body mode.
[baseTeleport]		Moves a popup panel to document.body so it escapes sticky/overflow-clipped ancestors.
Charts & timeline		
<base-trend-chart>		Rolling-average line chart with optional target band and area fill.
<base-bar-chart>		Category comparison bar chart — vertical or horizontal, per-point tone, stacked segments.
<base-scatter-chart>		Correlation chart between two metrics.
<base-histogram>		Distribution of one metric into touching bars/buckets.
<base-chart-frame>		Chart panel chrome — title/subtitle, export button, optional table/chart view toggle.
<base-state-heatmap>		Day × hour grid colored by dominant machine state.
<base-gantt-timeline>		24h per-row state segments — the detail view a heatmap cell expands into.
Table		
<base-table>	table/	Core data table — see §4.3–4.6 for the full feature list.
<base-table-cell>	table/	Renders one cell for its column's built-in kind (internal — used by base-table, not mounted directly).
<base-table-views>	table/	Saved/filtered view tab rail — pinned "All" plus saved views, Modified badge, Save/Update/Reset/Duplicate/Copy link.
<base-paginator>	table/	Standalone pagination control (also used inside the table); supports an unknown-total server mode.
<base-search-input>	table/	Debounced search box.

Selector	File	Purpose
<base-manage-columns>	table/	Gear-icon column visibility + drag-reorder panel; frozen columns lock at the top.
<base-checkbox-filter>	table/	Per-column value-filter dropdown: search, per-option record counts, "(No value)" row, optional sort.
<base-calendar-filter>	table/	Per-column date-range filter dropdown, with one-click relative presets (Last shift/24h/7d/30d).
<base-range-filter>	table/	Per-column numeric range filter dropdown with a mini distribution histogram.
Services		
BaseToastService		Imperative toast queue rendered by base-toast-host.
BaseThemeService		Applies the user's Light/Dark/High-contrast theme choice as <html data-theme>, persisted per user.
BaseDensityService		App-wide default table row density, set from the shell's utility-bar switcher.

All form controls expose a two-way [(value)] / [(checked)] model and implement `ControlValueAccessor`, so they also work with `ngModel` and `Reactive Forms` (`formControlName`) out of the box.

```
<base-text-input label="Tool ID" [(value)]="toolId" [clearable]="true"
  (enterPressed)="search($event)" />
<base-select label="Fab" [options]="fabOptions" [(value)]="fab" [searchable]="true" />
<base-datepicker label="Maintenance" [(value)]="date" [min]="minDate"
  [disabledDates]="noWeekends" />
<base-date-range-picker [(value)]="dateRange" (applied)="onRangeApplied($event)" />
<base-breadcrumbs [items]="crumbs" (itemClick)="onCrumb($event)" />
<base-tabs [tabs]="tabs" [(activeId)]="active" />
<base-modal [(open)]="show" title="Edit"> ... <div footer>...</div> </base-modal>
<base-drawer [(open)]="showInspector" side="right" width="460px" [showClose]="false">
  <fam-alarm-info-panel />
  <div footer class="w-full flex gap-2">
    <button class="btn-primary flex-1">View Event Log</button>
    <button class="btn-ghost flex-1 border border-slate-200">Export</button>
  </div>
</base-drawer>
```

Content projection gotcha: anything passed to `<base-drawer>`'s `[footer]` slot must be a direct child of `<base-drawer>` in the host template — Angular content projection does not reach through a nested component's own template, so footer buttons cannot live inside `<fam-alarm-info-panel>` itself if that component is what gets projected as the drawer's body.

4.2 Storybook — the interactive component catalog

`npm run storybook` (build-storybook for a static export) runs a full Storybook 10 instance (`@storybook/angular` + `addon-a11y` + `addon-docs`) documenting every Base component individually — the design-system deliverable that `/dev/base` (a single page wiring components together in a workflow) isn't meant to be.

How it maps to the design-system deliverable

Deliverable	Where it lives
Definition	The JSDoc comment above each component class, rendered on its Docs tab (via Compodoc).
Anatomy	The rendered story canvas.
Variants	Named stories per component (e.g. Primary, Secondary, Ghost, Danger).

Deliverable	Where it lives
States	Dedicated Disabled / Loading / Error / Empty stories, plus the Controls panel to toggle any input live.
Behavior	Component outputs, documented on the Docs tab; exercised live in the story.
Accessibility	@storybook/addon-a11y runs automated checks on every story (Accessibility tab).
Engineering handoff	This is the engineering artifact — the selector, inputs, and outputs shown are the real Angular API, not a mockup.

Organization (Storybook sidebar groups)

- **Base / Actions** — button, split button, segmented control, button group
- **Base / Forms** — text input, textarea, select, checkbox (+group), radio group, toggle, combobox, multi-select chips, selection cards, file upload, slider (+range), numeric stepper, OTP input, color picker, time picker, date picker, date range picker
- **Base / Data Display** — badge, tag, chip, avatar (+group), trend, KPI card, stat bar, sparkline, list item, accordion, card, divider, loading, skeleton, empty state, error page
- **Base / Navigation** — breadcrumbs, tabs, stepper, dropdown menu, context menu, notifications panel, global search
- **Base / Overlays** — modal, drawer, popover, hover card, alert, banner, progress bar, toast, tooltip
- **Base / Charts & Timeline** — trend chart, bar chart, scatter chart, histogram, chart frame, state heatmap, Gantt timeline
- **Base / Tables & Data** — table, table views, paginator, manage columns, checkbox/calendar/range filters
- **Foundations** — Color, Typography, Spacing & Sizing, Surfaces/Radius & Elevation, Grid & Layout, Iconography, Density & Themes, Token Architecture — the design tokens and system-level rules every component above is built from.
- **Widgets** — the WidgetConfig-driven layer every FAM screen composes pages from: KPI grid, chart, ranked list, table adapter, panel-chrome wrapper, and the 6-column responsive grid.
- **Layout** — app shell pieces with no service dependencies (currently: <fam-sidebar>).
- **Shared** — domain components built on top of Base (e.g. fam-state-legend).

Deliberately not storied yet: <fam-topbar>, the FAM/FCM/TQual/alarm-explorer route components, and domain chart components that inject() real app services (AuthService, ApiService, FilterStore, UptimeStore) rather than taking data as inputs — storying them needs stub providers or mocked HTTP, a separate effort from documenting the presentational component library. Every story imports the real component classes from src/app/base, so if a component's Angular API changes, its story updates to match or fails to compile — the catalog cannot drift from the code.

4.3 Table Model (models/table.model.ts)

All types consumed by <base-table>. A table is fully described by BaseColumnDef[] (dynamic columns, add/remove/reorder at runtime) and rows: T[] (any row shape) — 18 built-in cell kinds and a 17-type typed row-action registry, detailed below.

Cell kinds

Cell kind	Renders
text	Default — String(value).
number	Right-aligned, formatted via Intl.NumberFormat.

Cell kind	Renders
date	Formatted via Intl.DateTimeFormat.
datetime	Date + time, via Intl.DateTimeFormat (dateStyle + timeStyle).
sno	Auto row number (1, 2, 3...), page-aware when paginated.
array	value = unknown[] — deduped, comma-joined.
badge	Pill, colored via badgeClassMap.
dot	Status dot + text, colored via dotClassMap.
status-text	Colored bold text (no pill/dot), colored via textColorClassMap.
trend	▲/▼ % pill (BaseTrendComponent).
image	Thumbnail , value = URL.
progress	0–100 progress bar.
text-bar	Colored label with a thin proportional bar underneath.
sparkline	Mini line chart, value = number[].
link	Anchor; href from linkHref(row).
row-actions	Small icon buttons driven by a typed BaseRowAction[] registry (17 built-in types); more than [maxVisibleActions] (default 2) collapse into a "... " overflow menu, and/or freeform icon buttons; variant: 'button' renders a bordered ghost button instead of a plain icon.
heat-cell	Full-cell colored block for a value read against a threshold (via heatClassMap) — never color alone, the value text always shows alongside the tint.
template	Custom ng-template (BaseCellDirective).

The 17 built-in RowActionType values (each with an auto-resolved default icon, overridable per action): add, click, copy, delete, download (shows a live progress % while the row has a truthy fileProgress instead of the icon), edit, more, reset, run, upload, view, cancel, history, revert, apply, disable, enable. Each BaseRowAction also supports isDisabled(row) / isHidden(row) predicates per row. [readOnly]="true" on the table removes mutating action types (add/edit/delete/reset/revert/apply/disable/enable/cancel/upload) from the row entirely rather than merely disabling them.

number columns can set abbreviateNumbers: true (1.2M/84K, exact value in a tooltip); negative values get error-toned styling automatically. [showSummary]="true" pins a real <tfoot> aggregate row over the filtered row set, per column via summary: 'total' | 'mean' | 'median' | 'min' | 'max' | 'count' | 'outOfSpec' | 'none'.

Row grouping (groupBy) supports three header styles via groupHeaderStyle: 'accent' (default — indigo header with a row count), 'plain' (muted uppercase section divider, no count), and 'light' (white header, bold label, a rounded count pill labelled via groupCountLabel, and a solid action button). Return null from groupBy to leave a row ungrouped.

```
import { BaseTableComponent, BaseCellDirective, BaseColumnDef } from '../base';

columns: BaseColumnDef<Tool>[] = [
  { key: 'toolId', header: 'Tool', sticky: 'left', width: '120px', sortable: true, filterable: true },
  { key: 'photo', header: 'Photo', kind: 'image' },
  { key: 'uptime', header: 'Uptime %', kind: 'number', align: 'right', sortable: true },
  { key: 'status', header: 'Status', kind: 'badge',
    badgeClassMap: { UP: 'bg-emerald-50 text-emerald-600', DOWN: 'bg-red-50 text-red-600' } },
  { key: 'history', header: '7-day', kind: 'sparkline' },
  { key: 'actions', header: '', sticky: 'right', width: '90px' } // custom template below
];
```

```

<base-table class="panel block"
  [columns]="columns" [rows]="rows" trackKey="toolId"
  [showFilterRow]="true" [stickyHeader]="true" maxHeight="420px" minWidth="1100px"
  selectable="multiple"
  (rowClick)="open($event.row)"
  (sortChange)="onSort($event)"
  (pageChange)="onPage($event)"
  (filterChange)="onFilter($event)"
  (selectionChange)="selected = $event">

  <!-- CUSTOM CELL TEMPLATE: any content -- text, images, charts, buttons -->
  <ng-template baseCell="actions" let-row>
    <button class="btn-ghost" (click)="edit(row)">Edit</button>
  </ng-template>
</base-table>

```

Server-side mode

Set [serverSide]="true" and [totalItems]="totalFromApi". The table stops filtering/sorting/paginating internally and only emits (filterChange), (sortChange), (pageChange) — fetch data in the host and pass the new page via [rows].

Sticky columns

Set sticky: 'left' | 'right' and a fixed width on the column def. Pinned columns are automatically ordered to the edges and offsets are computed — header, left-frozen, and right-frozen all hold at once. Below 720px wide the right-frozen group automatically unfreezes (left stays pinned, since it's usually the row's own identity). Sticky columns are also treated as "frozen" by Manage Columns — locked, undraggable. Every other column stays user-pinnable at runtime via Manage Columns (Pinned left / Scrollable / Pinned right groups, drag or per-row pin buttons). A budget meter ([pinBudgetPercent], default 40) shows what % of total column width is pinned and is enforced on render — a view or manual pin that exceeds it un-pins from the right (outermost user-pinned column first) until back within budget, without touching the saved pin assignment. Applying emits (manageColumn), (pinChange), and (layoutChange) (BaseColumnLayout — order + visible keys + pinned as one fact, also available via getColumnLayout()/applyColumnLayout()).

Loading, error and empty states

[loading]="true" with no rows yet renders skeleton rows sized to the current density; with rows already present it dims them to 60% instead (a background refresh), and the paginator stays put either way. [error]="true" replaces the body with a recoverable error state ([errorMessage], (retry)) instead of the empty state. The empty state itself ([emptyKind]) auto-picks 'no-results' (with a one-click "Clear all filters" action) when a search/filter is active, or 'no-data' otherwise.

Inline edit

Opt in with [editableRows]="true" plus editable: true (+ editType, editOptions) on each column that should become a live control. A cell only swaps to its control while that row's row.isEditing is true — the host sets this, typically from a row action — and the table reflects state (amber tint while dirty, red via row.hasEditError on a failed save) and emits (cellEdit) on every change; it never commits anything itself. While any row is editing, sort/filter/page/Manage Columns all block with a dirty-count banner rather than risk losing track of an unsaved row.

Infinite scroll

[enableScroll]="true" + [maxHeight] emits (scrollEvent) as the container nears its top/bottom edge. [scrollTriggerPosition] picks which edge renders the loader/end row ('bottom', the default, for loading more; 'top' for loading older items above the current rows). [scrollLoading] shows a spinner row; [scrollEnd]="true" once there's nothing left replaces it with an end-of-list message.

Saved views

`<base-table-views>` is a separate, fully controlled component — it renders a tab rail (pinned "All" plus saved views, a Modified badge, Save/Update/Reset/Duplicate/Copy-link) and emits intent, but never inspects a view's state or re-applies it onto a table itself; the host owns the view list and derives "modified" by diffing its own live filter/sort/column snapshot against the active view's saved one. Update/Reset only render for a view the host can write back to; a modified shared/read-only view instead offers Reset alongside (duplicate), since Update isn't available on a view it doesn't own — the host responds by opening "Save view" pre-filled with the current live state. A view's column layout (order, visible keys, left/right pins) is captured via `getColumnLayout()` and restored via `applyColumnLayout()`; `<base-table>` also emits it on every column change via `(layoutChange)`.

Pagination

`[paginate]="true"` (the default) pages client-side at `[initialPageSize]` (default 10, options via `[pageSizeOptions]`). Changing the page size doesn't just reset to page 1: the table re-anchors to whichever page now contains the row that was first visible before the size changed and shows a note under the paginator saying where it landed — or, if that row can no longer be found, falls back to page 1 and says so instead of failing silently. Once the page count reaches `[pageEntryThreshold]` (default 10, also available on `<base-paginator>` directly), the paginator adds a direct "Go to page" field.

Whole-table save, drafts and leaving with work pending

`[editableRows]="true"` alone is enough for a whole-table save bar and a leave guard — `[draftId]` is optional and only adds cross-visit persistence. Every editable row moves through one of eight states: read-only, editing (clean), editing (dirty), saving, saved, save failed, parked as draft, and restored with conflict. While any row is dirty, a bar reads "N rows edited, not saved" with Navigate away (`confirmLeave()`), Discard changes (`discardAll()` — the only control that destroys work), and Save N changes (`saveAll()`, emitting one `(saveChanges)` request per dirty row); sort/filter/page/Manage Columns/view switching all block until it clears. Call `reportSaveResult()` once a save batch settles: successful rows become the new baseline and exit edit mode, failed rows keep their edits and dirty state, get an error tint, and stay editable for retry.

Setting `[draftId]` parks a leaving user's edits into `BaseEditDraftService` (localStorage, keyed by `[draftId]`, 7-day expiry, in-memory fallback if storage is blocked) instead of losing them — offered as "Keep draft" from the leave dialog (`confirmLeave()`), which also offers Stay on page / Save and leave / Discard). Reopening the table with a pending draft shows a banner to Review changes / Discard / Restore it; restoring re-applies its values as fresh edits, except any field whose server value has since changed, which is queued as a conflict for the user to resolve (Keep the server value / Restore mine as an edit).

Row-level visual indicators

`[rowIndicator]` colour-codes rows by a series/category in one of five forms — edge-marker (a leading-cell box-shadow strip), series-key (a swatch before a cell's content), magnitude-rule (a rule under a cell's content), inline-bar (a bar replacing a cell's content), and marker-column (an icon before a cell's content) — a table takes at most one. magnitude-rule/inline-bar share one scale per column (an explicit max, or the largest value across every row), printed as a footer legend so bars are never read against a maximum the user can't see.

The three kinds of percentage

A bare 45% doesn't say whether it's rising, falling, or what it belongs to: task progress (kind: 'task-progress' — something the operator started and can cancel, rendered as a ring + value + noun that only rises) is distinct from a measurement read against a threshold (kind: 'heat-cell' — can rise or fall, keeps the column's own precision) and a share of a set (kind: 'progress' or 'text-bar' — a bar against the column maximum).

4.4 Column filters & Manage Columns

filterable: true alone still keeps the classic text filter-row input. Set `filterKind`: 'checkbox' | 'calendar' | 'range' on a column to swap its header icon for a richer dropdown instead — `<base-table>` computes unique values and applies

the filter itself, no extra wiring needed beyond the column def.

filterKind	Component	Behavior
checkbox	<base-checkbox-filter>	Search box, checkbox list of unique column values with a per-option record count (evaluated against every other active filter), a synthetic "(No value)" row for nulls, optional Sort Asc/Desc radios, Clear link, Apply button.
calendar	<base-calendar-filter>	Start Date / End Date via two <base-datepicker>s (filterShowTime adds HH:MM boxes), one-click relative presets (Last shift/24 hours/7 days/30 days), Clear, Apply.
range	<base-range-filter>	Numeric From/To inputs with a mini distribution histogram of the column's values and inline "Enter Valid Range" validation; exclusive with all other filters/sorts once applied — the table shows a banner and blocks the other controls until it's cleared.

Manage Columns — set [manageColumns]="true" to show a gear icon on the first column header, opening a panel with search, Select All, a per-column visibility checkbox, and drag-to-reorder (Angular CDK drag-drop). Sticky (frozen) columns render locked at the top with no drag handle and cannot be hidden; a "last-standing" guard blocks unchecking the final visible column. Cancel discards the draft; Apply emits (manageColumn) with the new visible-key list. Non-locked columns stay grouped into Pinned left / Scrollable / Pinned right; any of them moves between groups by dragging it across, or via its per-row pin-left/pin-right buttons. A budget meter ([pinBudgetPercent] on <base-table>, default 40) shows the pinned width share and is enforced on render.

```
import { RowActionType, BaseRowAction } from '../..base';

rowActions: BaseRowAction<Tool>[] = [
  { type: 'edit', run: r => edit(r) },
  { type: 'delete', isDisabled: r => r.locked, run: r => remove(r) },
  { type: 'download', run: r => download(r) } // shows r.fileProgress% while > 0
];
```

```
<base-table [columns]="columns" [rows]="rows" [manageColumns]="true"
  (manageColumn)="visibleKeys = $event" (handleAction)="onAction($event)" />
```

4.5 Expandable nested child tables

Set [expandable]="true" to add a first-column toggle button that expands any row with children into a full nested <base-table> underneath it — its own columns ([childColumns]), its own row source ([childRowsOf], return null/undefined/[] to hide the toggle for a row with no children), and optionally its own paginator ([childPaginate]) and search box ([childShowSearch]). Toggling emits (expandChange) with { row, expanded }.

Two new directives support custom rendering inside the nested table: <ng-template baseChildCell="key"> (same context as baseCell, but targets a childColumns key) and <ng-template baseChildFooter> (presence-only content rendered below the nested table for every expanded row, e.g. an "Add Service Activity" button). Both are declared inside the outer <base-table> and forwarded automatically.

4.6 Template directives

The table recognizes five content-projection directives:

Directive	Required input	Purpose
ng-template[baseCell]	baseCell: string (column key)	Fully custom cell content for one column. Always wins over kind.

Directive	Required input	Purpose
ng-template[baseChildCell]	baseChildCell: string (childColumns key)	Same as baseCell, for a column of the nested expandable child table.
ng-template[baseHeaderCell]	baseHeaderCell: string (column key)	Custom header content for one column.
ng-template[baseActionTemplate]	baseActionTemplate: string (row-action type)	Overrides one row-action type's rendering everywhere it appears.
ng-template[baseChildFooter]	none (presence-only)	Content rendered below the nested child table for every expanded row.

baseCell exposes a typed template context: \$implicit (row), value, column, index. A template always wins over the column's built-in kind.

4.7 Date pickers

<base-datepicker> has an optional showTime mode — adds HH:MM boxes below the calendar grid and keeps the panel open after a date is picked so the time can be adjusted before closing. <base-date-range-picker> is a dropdown combining a quick-range sidebar (All / Last 1 / 7 / 30 Days / Custom range) with dual side-by-side month calendars, each with its own HH:MM time boxes. Its [(value)] (a DateRangeValue) only commits when Apply is clicked — Cancel reverts the draft; future dates are disabled (strikethrough) unless [disableFuture]="false".

```
<base-date-range-picker [(value)]="dateRange" [disableFuture]="true"
  (applied)="onRangeApplied($event)" />
```

4.8 Used Throughout the App

The whole application renders through this module — treat the features as live examples:

- **All dynamic tables (Uptime Analysis, Availability events/activities, Alarm Explorer)** — <base-table> + <base-paginator> via the fam-table-widget adapter, incl. grouped rows, group actions, highlight.
- **KPI grids / ranked lists** — <base-kpi-card>, <base-trend>, <base-progress-bar>.
- **Top bar & page filter bars** — the global topbar plus local filter bars (Alarm Explorer's Model/Fleet/Category/Duration, Fleet detail's Alarm ID/Tools/Category/SW Version, Alarm Events' Date Range/Recipe/Show/Duration, Uptime Availability's Tool-Level Analysis Filter) → <base-breadcrumbs> route trail + <base-select> filters.
- **Login** — <base-text-input formControlName> (ControlValueAccessor), <base-button>, <base-alert>.
- **Alarm Explorer pages** — <base-breadcrumbs> trails; the Tool page uses <base-search-input>, <base-select>, <base-button>, and opens its alarm inspector in a <base-drawer> instead of an inline side panel.

fam-kpi / fam-loading / fam-trend are deprecated wrappers delegating to base — new code should import from src/app/base directly. The fam-table-widget adapter currently exposes only a subset of <base-table> (grouped rows, group actions, highlight); column filters, Manage Columns, typed row actions, additional header rows, and infinite scroll are not wired through it yet — use <base-table> directly for those until the adapter is extended.

4.9 Full component reference

Every prop, event, content-projection slot, and behavior for every Base component — including all the items above — is now written up in full in a standalone reference: docs/Base-Module-Component-Guide.docx. This technical documentation covers the module at an architectural level; use that guide when you need the exact prop table for a specific component.

5. The Shared Module — App-Level Composition Layer (shared/)

Purpose: everything the app needs that isn't a generic reusable primitive — glue code, the dynamic/config-driven page system, and FAM-domain widgets. Built on top of base/, never the other way around.

5.1 components/ui.components.ts

KpiComponent (fam-kpi), LoadingComponent (fam-loading), TrendPillComponent (fam-trend) — deprecated thin wrappers that delegate 100% of rendering to `<base-kpi-card>` / `<base-loading>` / `<base-trend>`. StateLegendComponent (fam-state-legend) — a domain-specific legend for the machine states, colored from the FAM Tailwind theme tokens, with `withGap` and `withDayShift` flags for the heatmap and Activity Gantt.

5.2 dynamic/ — the config-driven widget system

Every screen is data (`WidgetConfig[]`), not a hand-written template. A feature component builds `computed<WidgetConfig[]>` and renders one line: `<fam-dynamic-page [widgets]="widgets()" />`.

type	Interface	Renders
kpi-grid	KpiGridWidget	Auto-fit grid of <code><fam-kpi></code> tiles.
chart	ChartWidget	Chart.js canvas (<code>chartType</code> , <code>data</code> , <code>options</code>), optional <code>onPointClick</code> drill-down.
table	TableWidget<Row>	Sortable columns (<code>text/mono/badge/dot/trend/progress/text-bar/row-actions</code> cell kinds), row grouping, pagination, row actions.
ranked-list	RankedListWidget	Ranked rows with <code>title/subtitle/value/trend pill/progress bar</code> , clickable.
component	ComponentWidget	Arbitrary Angular component, via direct component: <code>Type<...></code> or name + registry.

All widget types share `WidgetBase`: `id`, `title`, `titlePrefix`, `subtitle`, `badge`, `colSpan` (1–6 on a 6-column grid), `legend`, `actions`, `actionsLabel/note`, and `frameless`, plus header extras: `dateRange`, `tabs`, `toggle`, `search`, and `collapsible/collapsed/onToggleCollapse`.

5.3 utils/

`csv.util.ts` — `downloadCsv(filename, rows)` for CSV export buttons. `category-colors.util.ts` — `buildCategoryColorMap` / `buildCategoryTextColorMap`: stable Tailwind color assignment per category in first-seen order, used by the downtime-category and alarm-category color coding.

6. The Core Module — Data & Infrastructure Layer (core/)

- **models/models.ts** — every TypeScript interface that is the contract between mock data and a real backend, grouped by feature area (Global, Up-Time Analysis, Up-Time Availability, Alarm Explorer).
- **mock-api/** — `mock-data.ts` (seeded, deterministic `build...()` generators) + `mock-api.interceptor.ts` (routes `/api/**` to a generator with artificial latency).
- **services/api.service.ts** — `ApiService`, the single HTTP gateway — one typed method per endpoint.
- **state/** — `base.store.ts` primitives (`createQuery`, `createPagination`, `createListFilter`) plus `filter.store.ts`, `uptime.store.ts`, `alarm.store.ts`, and `segment-derivation.util.ts` (derives Gantt/heatmap/event rows from one raw feed).
- **auth/** — `AuthService` (signal-based session, dummy JWT), `authGuard/authChildGuard`, `authInterceptor`.
- **constants/** — `app.constants.ts` (routes, nav groups, brand text, auth config, UI copy) and `component-catalog.ts` (`COMPONENT_CATALOG`, the runtime index rendered at `/dev/components`).

```
@Injectable({ providedIn: 'root' })
export class MyModuleStore {
  private api = inject(ApiService);
  readonly detailQuery = createQuery(
    (id: string) => this.api.getDetail(id),
    { cacheTtlMs: 300_000 }
  );
  readonly rows = computed(() => this.detailQuery.data()?.rows ?? []);
  readonly pager = createPagination(this.rows, 20);
}
```

7. Layout — The App Shell (layout/)

Not a "feature" — the persistent frame every routed screen renders inside. `shell.component.ts` (`fam-shell`) is a flex layout of `<fam-sidebar>` + a right column of `<fam-topbar>`, `<router-outlet>`, and a footer, guarded by `authGuard/authChildGuard`. `sidebar.component.ts` (`fam-sidebar`) renders `NAV_GROUPS` as `routerLinks`. `topbar.component.ts` (`fam-topbar`) builds `<base-breadcrumbs>` from the current URL, a Date Range chip, `<base-select>` fleet/duration pickers bound to `FilterStore`, and a user-avatar popup.

8. Features & Routing (features/)

```

/login                               LoginComponent (public)
/ (ShellComponent, authGuard)
├ ''                                → redirect to /fleet-availability/up-
time/analysis
├ fleet-availability/up-time/analysis    UptimeAnalysisComponent
├ fleet-availability/up-time/availability UptimeAvailabilityComponent
├   └ ''                                → redirect to ../heatmap
├     └ heatmap / gantt / events / activities Tab components
├ alarm-explorer                      AlarmHomeComponent
├ alarm-explorer/fleet/:fleetId        FleetDetailComponent
├ alarm-explorer/fleet/:fleetId/tool/:toolId ToolAlarmsComponent
├ alarm-explorer/fleet/:fleetId/tool/:toolId/alarm/:alarmId AlarmEventsComponent
├ fleet-configuration, fleet-productivity, tqual,
  my-reports, innovation-lab, engineering-utilities PlaceholderComponent (all six)
├ dev/components                      ComponentCatalogComponent
├ dev/base                            BasePlaygroundComponent
**                                    → redirect to /

```

Route path/title strings all come from APP_ROUTE_PATHS/ROUTE_TITLES in core/constants/app.constants.ts. The dev/base playground was extended alongside each Base Module feature addition (expandable rows, filters, Manage Columns, date-range picker) so every new capability is exercised live there too.

Area	Files	Store	Notes
Auth	features/auth/login.component.ts	AuthService directly	Reactive form, base-text-input/base-button/base-alert.
Up-Time Analysis	features/uptime-analysis/*.component.ts	UptimeStore	Uptime/Downtime toggle trend chart, collapsible breakdown table, top-10 unavailable chart, downtime table, CSV export.
Up-Time Availability	features/uptime-availability/*.component.ts (5 files)	UptimeStore	Page shell + 4 routed tabs + Tool-Level Analysis Filter bar; heatmap/gantt/events share one derived feed.
Alarm Explorer	features/alarm-explorer/*.component.ts (5 files)	AlarmStore	4-level drill-down (home → fleet → tool → alarm events) + info drawer.
Placeholder	features/placeholder/placeholder.component.ts	none	"Coming soon" screen, reused across 6 unbuilt routes.
Dev tools	features/dev/*.component.ts	COMPONENT_CATALOG / none	/dev/components catalog browser, /dev/base live base-module playground.

9. Reference — Domain Models

See `core/models/models.ts` for the full list. Grouped by feature area:

- **Global** — `GlobalFilters`, `ToolState`, `AlarmCategory`, `Severity`
- **Up+Time Analysis** — `WeeklyUptimePoint`, `UptimeBreakdownRow`, `UnavailableTool`, `DowntimeCategory`, `UptimeAnalysisResponse`, `UptimeTrendResponse`
- **Up+Time Availability** — `AvailabilityKpis`, `FleetTrendPoint`, `HeatmapDay`, `StateTotals`, `GanttSegment`, `GanttDay`, `ToolEvent`, `AvailabilityResponse`, `StateSegment`, `SegmentActivity`
- **Alarm Explorer** — `FleetAlarmSummary`, `AlarmVolumeWeek`, `ToolAlarmSummary`, `AlarmDefinition`, `AlarmEvent`, `AlarmHomeResponse`, `FleetAlarmDetailResponse`, `ToolAlarmDetailResponse`, `AlarmEventsResponse`

10. Adding a New Module — Checklist

Repeatable process for turning a placeholder module — or any new screen — into a real, demoable component backed by mock data:

- **1. Model** — add response/row interfaces to `core/models/models.ts`.
- **2. Mock data** — add a `build...()` generator to `core/mock-api/mock-data.ts`.
- **3. Mock route** — add the matching path branch in `mock-api.interceptor.ts`.
- **4. Service** — add a typed method to `ApiService` (`core/services/api.service.ts`).
- **5. Store** — add or extend a signal Store under `core/state/`, composing `createQuery` (+ `createPagination/createListFilter` if the screen needs paging or client-side filtering).
- **6. Component** — build the screen, preferring `WidgetConfig/<fam-dynamic-page>` composition over bespoke templates. Need a bespoke visual? `registerWidget('my-widget', MyComponent)` in `register-widgets.ts`, or render it directly inside a `<base-drawer>` if it needs to overlay the whole page.
- **7. Route** — replace the placeholder route entry in `app.routes.ts` with the real `loadComponent`, and update `NAV_GROUPS/ROUTE_TITLES` if needed.
- **8. Catalog (optional but expected)** — add an entry to `COMPONENT_CATALOG` in `core/constants/component-catalog.ts` so it shows up at `/dev/components`.
- **9. Storybook (optional, recommended for new base-* components)** — add a `*.stories.ts` file under `src/stories/` so the component gets a Docs tab, live Controls, and automated accessibility checks alongside the rest of the catalog.

This keeps every delivered component demoable standalone (no backend needed) and makes the eventual mock-to-real-API swap a mechanical, low-risk change.